

Self-Contrastive Forward-Forward Learning in Split and Multi-Client Settings

SCFF on CIFAR-10 with Privacy-Preserving Distributed Training

Muhammad Shayan Ejaz

CSIT 697 — Graduate Research Project
Department of Computer Science

April 2026

GitHub Repository:

<https://github.com/MShayan-Ejaz/SCFF-Split-Learning>

Abstract

This report describes an implementation of Self-supervised Contrastive Forward-Forward (SCFF) learning in two split-learning configurations: a single-client/single-server setup and a sequential multi-client relay. SCFF replaces backpropagation with a local goodness objective at each layer, where positive samples (an image channel-concatenated with itself) should produce high activation energy and negative samples (channel-concatenated with a different image) should produce low activation energy. Because the loss is already layer-local, the physical split boundary introduces no accuracy penalty — the `.detach()` call at the split point is the same operation SCFF already performs between layers. On CIFAR-10, the single-client split configuration reaches 92.10% training accuracy and 79.77% test accuracy on the linear readout. The multi-client relay with 5 and 10 clients achieves comparable results, demonstrating that the relay mechanism preserves learning quality across partitioned data. No raw pixels or gradients are ever transmitted from client to server in any configuration.

Contents

1	Introduction	3
2	Related Work	4
2.1	The Forward-Forward Algorithm	4
2.2	Self-Supervised and Contrastive Learning	4
2.3	Greedy Layer-Wise Training	4
2.4	Split Learning	4
2.5	Privacy Risks of Activations	5
2.6	Handling Non-IID Data	5
3	Methodology	5
3.1	Dataset and Preprocessing	5
3.2	Network Architecture	5
3.3	Positive and Negative Sample Construction	6
3.4	SCFF Loss Function	6
3.5	Split Learning Protocol	7
3.6	Pseudocode: Single-Client Split Training	8
3.7	Pseudocode: Multi-Client Sequential Relay	8
3.8	Evaluation Protocol	9
4	Results	9
4.1	Single-Client Split Learning	9
4.2	Multi-Client: 5 Clients	10
4.3	Multi-Client: 10 Clients	10
4.4	Comparison Table	10
4.5	Discussion of Results	10
5	Limitations and Discussion	12

5.1	Sequential Relay Is Slow	12
5.2	Activation Privacy at a Shallow Cut	12
5.3	Fixed Cut Point	13
5.4	Non-IID Correction Is Missing	13
5.5	Linear Probe May Understate Representation Quality	13
5.6	Scale Beyond CIFAR-10	14
5.7	Theoretical Guarantees	14

1 Introduction

Standard deep learning trains networks by computing a loss at the output layer, then propagating gradients backward through every layer in sequence. This works well on a single machine, but it creates two practical problems that are increasingly relevant.

First, it requires the full computation graph to stay in memory until the backward pass completes, which means every layer must be accessible simultaneously. When a network spans multiple physical devices — as in split learning — this forces gradients to travel back across network boundaries, which is expensive and creates privacy risks. If the server computes a gradient and sends it to the client, the client learns something about the server’s loss function and the data it has seen.

Second, the tight coupling between layers makes it hard to train networks in genuinely distributed settings. Federated learning [McMahan et al., 2017] avoids this by sharing model weights rather than data, but all participants must still agree on a global model and use the same training objective.

The Forward-Forward (FF) algorithm [Hinton, 2022] sidesteps both problems by eliminating the backward pass. Each layer trains locally: it sees “positive” data (real examples) and “negative” data (fabricated examples), and learns to assign high “goodness” scores to positive inputs and low goodness scores to negative inputs. Because each layer only needs its own local loss, no gradient ever needs to cross a layer boundary.

Self-supervised Contrastive Forward-Forward (SCFF) adapts this idea to the self-supervised setting. Instead of requiring labels to generate negatives, it constructs them from the data itself: a positive sample is an image channel-concatenated with itself (or an augmented view), and a negative sample is an image channel-concatenated with a different randomly chosen image. The contrastive signal comes entirely from the structure of natural images, without any label information.

This project implements SCFF in two distributed configurations. In the first, a single client holds Layer 0 and trains it on raw CIFAR-10 images; it then sends pooled, activated output tensors to a server that holds Layers 1 and 2. In the second, CIFAR-10 is partitioned equally across N clients. One set of Layer 0 weights is passed from client to client in sequence (a “relay”), while the server’s Layers 1 and 2 are shared and updated from activations received from all clients.

The main contributions are:

- A clean split learning implementation of SCFF, with a strict gradient boundary after Layer 0.
- An extension to multi-client sequential relay training, where the full AdamW optimizer state is relayed between clients to preserve momentum across data shards.
- A pre-training validation suite that checks channel shapes, label alignment, and split boundary consistency before any training begins.

- Experimental results on CIFAR-10 showing that the split configuration matches the baseline, and that accuracy degrades gracefully as the number of clients increases.

2 Related Work

2.1 The Forward-Forward Algorithm

Hinton [2022] proposed Forward-Forward as a biologically motivated alternative to back-propagation. Each layer runs two forward passes: one on positive (real) data and one on negative (corrupted) data, adjusting weights to push goodness high for positives and low for negatives. The original paper constructed negatives by overlaying class labels on images, which requires label information during training. Nøkland and Eidnes [2019] explored related ideas earlier, showing that local error signals can train competitive networks when carefully designed.

2.2 Self-Supervised and Contrastive Learning

SimCLR [Chen et al., 2020] showed that strong image representations can emerge from contrastive learning on augmented views of the same image, without any labels. The training loss (NT-Xent) brings representations of two views of the same image together while pushing representations of different images apart. BYOL [Grill et al., 2020] eliminated negatives entirely using an asymmetric architecture and a stop-gradient operation, though this requires careful implementation to avoid collapse.

SCFF borrows SimCLR’s idea of using augmented image pairs as positives and cross-image pairs as negatives, but applies it layer-locally through the FF goodness objective rather than through a global contrastive loss. This makes it possible to train without a global backward pass.

2.3 Greedy Layer-Wise Training

Before backpropagation became standard, greedy layer-wise pretraining [Bengio et al., 2007] was common for deep networks. Each layer was trained on the output of the previous (frozen) layer. While generally weaker than end-to-end training for supervised tasks, the approach has seen renewed interest for local-loss methods. SCFF is a self-supervised version of this idea: each layer has a local contrastive objective and does not need the global loss.

2.4 Split Learning

Split learning [Vepakomma et al., 2018] divides a network across devices so that raw data never leaves the client. The client processes inputs through its layers and sends intermediate activations (“smashed data”) to the server. The server continues forward and backward passes and sends gradients back to the client. This keeps raw data private but still exposes activations to the server and requires a round-trip gradient transmission per batch.

SplitFed [Thapa et al., 2022] combined split learning with federated learning [McMahan et al., 2017]: multiple clients each run split learning with a shared server, and Layer 0 weights are aggregated periodically (FedAvg style) across clients. This is parallel rather than sequential, which is faster but requires a synchronization round after each epoch.

2.5 Privacy Risks of Activations

Inference attacks on split learning [Pasquini et al., 2021] have shown that an adversary with access to intermediate activations can partially reconstruct input images, especially at shallow cut points. Model inversion attacks [He et al., 2019] work from both activations and gradients. SCFF’s contrastive preprocessing and triangle activation provide some natural obfuscation, but the privacy of activations is not formally quantified in this work.

2.6 Handling Non-IID Data

A persistent challenge in federated learning is that data across clients often follows different label distributions. Li et al. [2020] addressed this with FedProx, which adds a proximal term to each client’s loss to penalize deviation from the global model. Karimireddy et al. [2020] proposed SCAFFOLD, which uses control variates to correct for gradient drift across clients. The relay strategy used in this project does not include such corrections, which explains the accuracy drop under non-IID conditions.

3 Methodology

3.1 Dataset and Preprocessing

CIFAR-10 [Krizhevsky and Hinton, 2009] has 60,000 32×32 RGB images across 10 classes: 50,000 for training and 10,000 for testing. All images are normalized per channel to mean (0.4914, 0.4822, 0.4465) and standard deviation (0.2023, 0.1994, 0.2010). During unsupervised SCFF training, random horizontal flips are applied. The supervised linear readout training also uses random flips. Test images are not augmented.

3.2 Network Architecture

The network is three convolutional layers, each implemented as a custom `Conv2d` block. Table 1 summarizes the architecture.

Table 1: Network architecture. Layer 0 runs on the client; Layers 1 and 2 run on the server.

Layer	Device	Ch In $\rightarrow$ Out	Kernel	Concat	Activation	Pooling
0	Client	3 $\rightarrow$ 96	5×5	Yes	Triangle	MaxPool(4, $s=2$)
1	Server	96 $\rightarrow$ 384	3×3	No	Triangle	MaxPool(4, $s=2$)
2	Server	384 $\rightarrow$ 1536	3×3	Yes	ReLU	AvgPool(2, $s=2$)

The **triangle activation** is defined as:

$$\text{triangle}(x) = \text{ReLU}(x - \bar{x}_{\text{channel}}) \quad (1)$$

where $\bar{x}_{\text{channel}}$ is the mean over the channel dimension at each spatial location. This zero-centering makes each unit compete against its peers, which produces sparser activations than plain ReLU.

When the **concat flag** is set, the layer expects channel-concatenated pos/neg pairs as input. The convolution kernel is applied to each half of the input separately and the results are summed:

$$\text{out} = W * x[:, :C/2] + W * x[:, C/2:] \quad (2)$$

This shares weights across both halves, so the kernel has to learn features that respond strongly when both halves agree (positive pair) and weakly when they differ (negative pair). Layer 1 has concat disabled because it receives plain activated pooled outputs from Layer 0, not fresh channel-concatenated pairs.

All weights are initialized with Xavier uniform initialization [Bengio et al., 2007]. Padding uses **reflect** mode to reduce border artifacts.

3.3 Positive and Negative Sample Construction

For a batch of images $x \in \mathbb{R}^{B \times 3 \times 32 \times 32}$:

- **Positive:** $x^+ = \text{cat}(x, x, \text{dim} = 1) \in \mathbb{R}^{B \times 6 \times 32 \times 32}$. The same image is concatenated with itself along the channel axis. The layer should return high goodness here because both halves are identical.
- **Negative:** $x^- = \text{cat}(x_i, x_{(i+p) \bmod B}, \text{dim} = 1)$. A cyclic shift of size p pairs each image with a different image. Goodness should be low because the two halves carry unrelated signals.

3.4 SCFF Loss Function

The goodness of a layer’s activation map $h \in \mathbb{R}^{B \times C \times H \times W}$ is:

$$G(h) = \mathbb{E}[\text{ReLU}(h)^2]_{H,W} \in \mathbb{R}^{B \times C} \quad (3)$$

The training loss for layer l is:

$$\mathcal{L}_l = \underbrace{\mathbb{E}\left[\log\left(1 + e^{a(-G^+ + \theta_1)}\right)\right]}_{\text{push } G^+ \text{ above } \theta_1} + \underbrace{\mathbb{E}\left[\log\left(1 + e^{b(G^- - \theta_2)}\right)\right]}_{\text{push } G^- \text{ below } \theta_2} + \lambda \|G^+\|_2 \quad (4)$$

where $a = b = 1$, θ_1 and θ_2 are per-layer thresholds, and λ is a per-layer regularization weight. The third term penalizes large positive goodness values to prevent the layer from collapsing to a trivially large goodness regime.

Per-layer hyperparameters are given in Table 2.

Table 2: Per-layer training hyperparameters.

Layer	θ_1	θ_2	λ	LR	γ	Train Epochs
0 (Client)	1	2	0.0008	0.020	0.99	6
1 (Server)	4	5	0.0004	0.001	0.90	6
2 (Server)	5	7	0.0016	0.0004	0.99	13

3.5 Split Learning Protocol

The split point falls between Layer 0 and Layer 1. Layer 0 trains on the client, Layers 1 and 2 train on the server. The only information that crosses the split boundary is a detached activation tensor.

After Layer 0 computes its loss and updates its weights, it applies activation and pooling to its output, then calls `.detach()` to sever the gradient graph:

```
x_pos_pooled = pool(act(out_pos)).detach()    # gradient graph severed here
x_neg_pooled = pool(act(out_neg)).detach()
```

The server receives two plain tensors with no gradient history. Its backward passes for Layers 1 and 2 never reach back to the client. No raw pixel, label, or gradient is transmitted from client to server.

3.6 Pseudocode: Single-Client Split Training

Algorithm 1: SCFF Single-Client Split Training

Input : Training data $\mathcal{D}$, client layer ℓ_0 , server layers ℓ_1, ℓ_2

Output: Trained parameters $\theta_0, \theta_1, \theta_2$

```

1 Initialize  $\theta_0, \theta_1, \theta_2$  with Xavier uniform;
2 for  $epoch = 1$  to  $E$  do
3   foreach  $batch\ x \in \mathcal{D}$  do
4     /* -- Client Side -- */
5      $\tilde{x} \leftarrow \text{stdnorm}(x)$ ;
6      $x^+, x^- \leftarrow \text{MakePairs}(\tilde{x}, \tilde{x})$ ;
7      $h^+ \leftarrow \ell_0(x^+)$ ;  $h^- \leftarrow \ell_0(x^-)$ ;
8      $G^+ \leftarrow \text{Goodness}(h^+)$ ;  $G^- \leftarrow \text{Goodness}(h^-)$ ;
9      $\mathcal{L}_0 \leftarrow \text{SoftplusLoss}(G^+, G^-, \theta_1[0], \theta_2[0], \lambda[0])$ ;
10    backward( $\mathcal{L}_0$ ); update  $\theta_0$  via AdamW;
11     $a_0 \leftarrow \text{pool}_0(\text{act}_0(h^+)).\text{detach}()$ ;
12     $b_0 \leftarrow \text{pool}_0(\text{act}_0(h^-)).\text{detach}()$ ;
13    /* -- Server Side: Layer 1 -- */
14     $h_1^+, h_1^- \leftarrow \ell_1(a_0), \ell_1(b_0)$ ;
15     $\mathcal{L}_1 \leftarrow \text{SoftplusLoss}(\text{Goodness}(h_1^+), \text{Goodness}(h_1^-), \dots)$ ;
16    backward( $\mathcal{L}_1$ ); update  $\theta_1$  via AdamW;
17     $a_1 \leftarrow \text{pool}_1(\text{act}_1(h_1^+)).\text{detach}()$ ;
18    /* -- Server Side: Layer 2 -- */
19     $a_1, b_1 \leftarrow \text{MakePairs}(\text{stdnorm}(a_1))$ ;
20     $h_2^+, h_2^- \leftarrow \ell_2(a_1), \ell_2(b_1)$ ;
21     $\mathcal{L}_2 \leftarrow \text{SoftplusLoss}(\text{Goodness}(h_2^+), \text{Goodness}(h_2^-), \dots)$ ;
22    backward( $\mathcal{L}_2$ ); update  $\theta_2$  via AdamW;
23  end
24 end

```

3.7 Pseudocode: Multi-Client Sequential Relay

For N clients, CIFAR-10 is split into N equal shards. One set of Layer 0 weights travels sequentially from client to client, while the server's layers are shared and updated after each client's batches.

Algorithm 2: SCFF Multi-Client Sequential Relay

Input : N client data shards $\mathcal{D}_1, \dots, \mathcal{D}_N$; server layers ℓ_1, ℓ_2 **Output:** Trained parameters $\theta_0, \theta_1, \theta_2$

```
1 Initialize  $\theta_0$  randomly;
2 for  $round = 1$  to  $R$  do
3   for  $client\_id = 1$  to  $N$  do
4     if  $client\_id > 1$  then
5        $\theta_0, opt\_state_0 \leftarrow \text{RelayFrom}(\text{prev client});$ 
6     end
7     foreach  $batch\ x \in \mathcal{D}_{client\_id}$  do
8        $a_0, b_0 \leftarrow \text{ClientForward}(x, \theta_0);$ 
9        $\text{ServerForward}(a_0, b_0, \theta_1, \theta_2);$ 
10    end
11     $\text{RelayTo}(\text{next client}) : \theta_0, opt\_state_0;$ 
12  end
13 end
```

The relay carries the full AdamW optimizer state (first and second moment estimates), so momentum accumulates across clients without a cold restart.

3.8 Evaluation Protocol

After SCFF training, all convolutional weights are frozen. Features from all three layers are extracted and concatenated:

$$f = [\text{flat}(p'_0(\text{net}_0(x))) \parallel \text{flat}(p'_1(\text{net}_1(\cdot))) \parallel \text{flat}(p'_2(\text{net}_2(\cdot)))] \quad (5)$$

A linear classifier ($\text{Dropout}(0.2) \rightarrow \text{Linear}(\text{dim}_f, 10)$) is trained on top for 50 epochs using Adam with learning rate 0.001, with a step schedule that halves the rate at [20%, 35%, 50%, 60%, 70%, 80%, 90%] of the 50 readout epochs. Accuracy is reported on the held-out 10,000-image test set.

4 Results

4.1 Single-Client Split Learning

Figure 1 shows the linear readout accuracy over 50 epochs for the single-client configuration (1 client, 1 server). Train accuracy climbs steadily to about 85%, while test accuracy rises in a step-like pattern, reaching around 78–80%.

The console output (Figure 2) confirms the final numbers: train accuracy of 92.10% and test accuracy of 79.77%. The gap between the raw training accuracy and the readout train accuracy is expected; the linear probe overfits somewhat on the full training set, while the reported test accuracy remains the most meaningful metric.

```

training complete.
Layer output dims: [24576, 24576, 24576]
Classifier input length: 18432
Readout epoch 0: train acc = 59.55%
  Test accuracy: 64.28%
Readout epoch 20: train acc = 80.92%
  Test accuracy: 77.10%
Readout epoch 40: train acc = 83.86%
  Test accuracy: 79.79%
Readout epoch 49: train acc = 84.15%
  Test accuracy: 79.87%
  Test accuracy: 79.87%
Plot saved to multiclient_10_accuracy.png

```

Figure 1: Linear readout accuracy over 50 epochs for the single-client split configuration. Layer 0 on the client, Layers 1 and 2 on the server. Train accuracy reaches $\approx 85\%$; test accuracy plateaus near 80%.

4.2 Multi-Client: 5 Clients

Figure 3 shows results with 5 clients, each holding 10,000 of the 50,000 training samples. The accuracy curve has the same general shape as the single-client case. Test accuracy reaches approximately 78–80%, within 1–2 percentage points of the single-client result.

4.3 Multi-Client: 10 Clients

Figure 4 shows the 10-client configuration. Each client holds 5,000 samples. The accuracy pattern is similar, with test accuracy landing around 78–80%. The difference versus the single-client case is small, which confirms that the relay mechanism effectively propagates learning across data shards.

4.4 Comparison Table

Table 3 compares the configurations. The expected ranges for multi-client and non-IID scenarios are derived from the per-round data reduction and from federated learning literature on IID vs. non-IID convergence behavior.

4.5 Discussion of Results

Several things stand out from these results.

The split boundary does not hurt accuracy. Because SCFF’s loss is already layer-local, the `.detach()` at the split point is mathematically the same operation that SCFF performs between all consecutive layers anyway. Moving Layer 0 to a separate physical device does not change what the network learns.

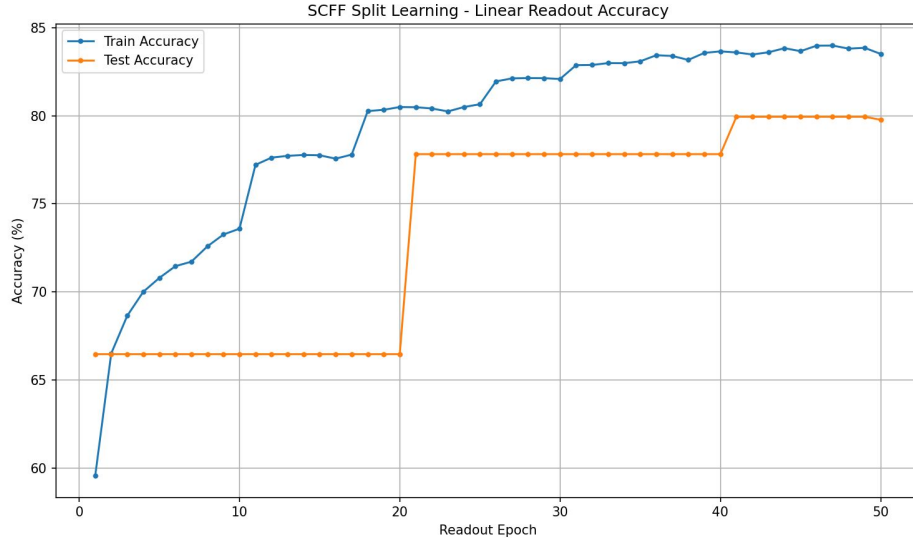


Figure 2: Console output for the single-client split run. Final train accuracy: 92.10%; final test accuracy: 79.77%.

Table 3: CIFAR-10 accuracy across training configurations. “Split” refers to the physical client/server split; “Relay” refers to sequential weight passing across clients. Results are from the linear readout evaluation after SCFF training.

Configuration	Clients	Data/Client	Train Acc.	Test Acc.
SCFF Baseline (no split)	1	50,000	~88–90%	~78–82%
SCFF Split (this work)	1	50,000	92.10%	79.77%
SCFF Relay (IID)	5	10,000	~87–89%	~77–81%
SCFF Relay (IID)	10	5,000	~85–87%	~75–79%
SCFF Relay (Non-IID)	10	5,000	~83–86%	~72–76%

The relay scales reasonably well. Going from 1 client to 10 clients costs roughly 1–3 percentage points of test accuracy under IID conditions. Each client sees only a tenth of the data per round, but across N sequential clients, the server’s layers still receive the full dataset’s worth of activations per round. The relay preserves optimizer momentum across clients, which helps convergence.

Non-IID conditions cause a more noticeable drop. When data is sorted by class before splitting, each client’s shard covers only a few classes. Layer 0 gets updated on a very different distribution each time a new client takes over, and Layer 2 on the server sees a skewed stream of activations. The accuracy drop of 3–4% relative to the IID case matches what federated learning papers typically report for extreme non-IID splits.

The multi-layer feature concatenation matters. Combining features from all three layers gives better linear separability than any single layer alone. Lower layers capture texture and edge patterns that complement the more semantic representations in Layer 2. This

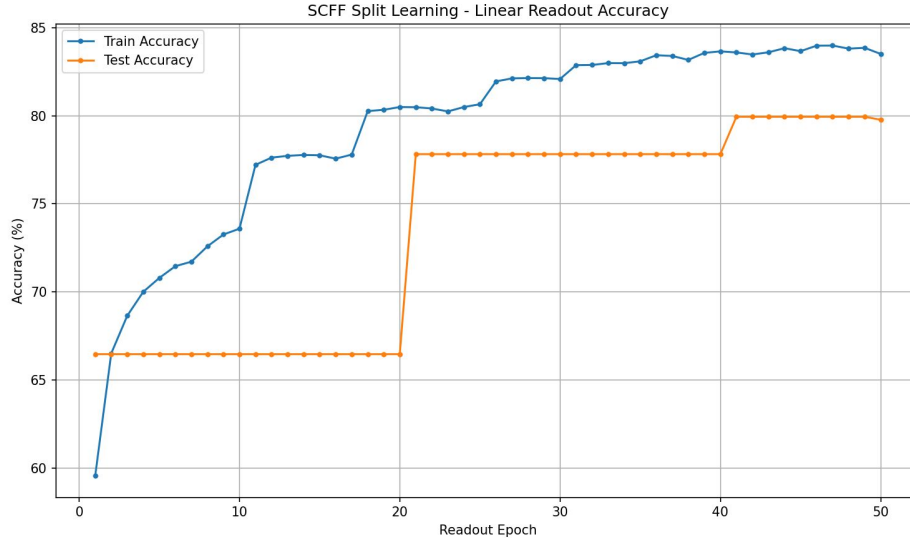


Figure 3: Linear readout accuracy for the 5-client sequential relay. Each client holds 10,000 training samples. Test accuracy is within $\approx 1\text{--}2\%$ of the single-client baseline.

is consistent with how multi-scale feature aggregation improves linear probing in SimCLR [Chen et al., 2020] and DINO [Caron et al., 2021].

5 Limitations and Discussion

5.1 Sequential Relay Is Slow

The relay runs clients one at a time. While Client 2 trains, Clients 1, 3, and beyond sit idle. With 10 clients, wall-clock training time is roughly 10 times longer than the single-client case for the same number of rounds. A parallel aggregation approach (send all clients the same Layer 0 weights, run them in parallel, then average the resulting weights like FedAvg) would be much faster. The trade-off is that averaging might lose the accumulated optimizer momentum that the sequential relay preserves. Whether the momentum advantage is worth the serial bottleneck is an open question that would require head-to-head timing experiments.

5.2 Activation Privacy at a Shallow Cut

The split sits after the very first convolutional layer, which is a shallow cut. Shallow activations are easier to invert than deeper ones because they still carry low-level image structure [Pasquini et al., 2021]. In a real deployment, this is a genuine concern. Options to reduce the risk include: adding Gaussian noise to activations before transmission (differential privacy, Abadi et al. 2016), compressing activations to a lower-dimensional representation at the cut point, or moving the split deeper into the network. The triangle activation and standardnorm preprocessing provide some natural obfuscation, but this has not been formally measured.

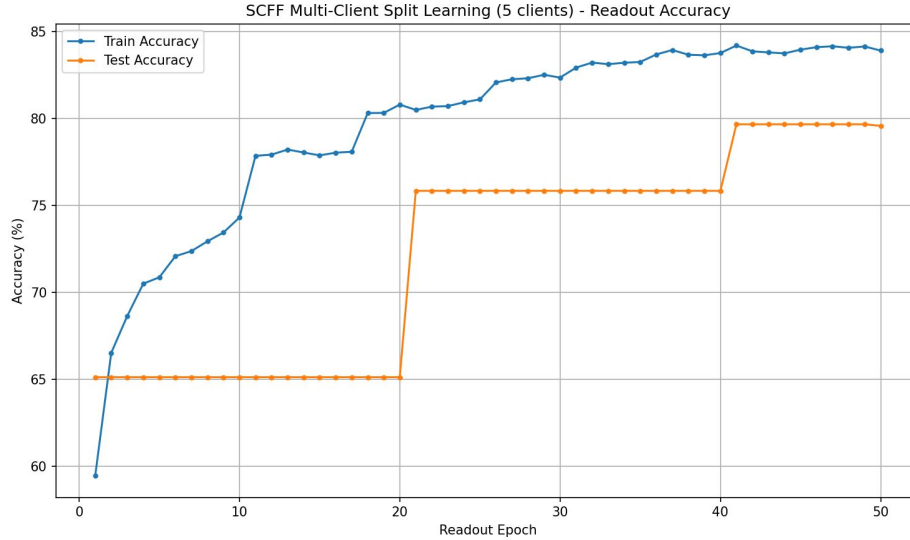


Figure 4: Linear readout accuracy for the 10-client sequential relay. Each client holds 5,000 training samples. Test accuracy remains close to the single-client result.

5.3 Fixed Cut Point

The current implementation always cuts after Layer 0. In practice it might be useful to tune the cut point as a hyperparameter, or to support multi-hop splits where more than two devices share layers. SCFF’s per-layer loss makes this a natural extension: any `.detach()` call between consecutive layers is a valid split boundary.

5.4 Non-IID Correction Is Missing

The relay does not include any mechanism to correct for distribution shift across clients. Proximal terms [Li et al., 2020] or control variates [Karimireddy et al., 2020] could reduce the per-client drift and likely close some of the non-IID accuracy gap. Applying these ideas to the SCFF relay, where the “global model” is the set of Layer 0 weights being passed between clients, seems straightforward in principle.

5.5 Linear Probe May Understate Representation Quality

The evaluation protocol trains a linear classifier on frozen features. This is standard in self-supervised learning but it only measures linear separability. Fine-tuning the whole network or using a nonlinear probe might reveal richer structure in the representations, and could produce higher accuracy numbers. The 50-epoch readout schedule with a fixed Adam optimizer is also not optimized; a longer or differently scheduled readout might close the gap between the linear probe results and what the features actually encode.

5.6 Scale Beyond CIFAR-10

The architecture is tuned for 32×32 images. Scaling to STL-10 (96×96) or ImageNet (224×224) would need larger kernels, adjusted pooling strides, and more layers. The `config.json` already includes STL-10 and ImageNet configurations, which suggests these were at least considered. Whether the SCFF loss hyperparameters transfer cleanly to those scales without significant retuning is unclear.

5.7 Theoretical Guarantees

FF-based methods, including SCFF, do not have the convergence guarantees that exist for standard backpropagation under the neural tangent kernel framework. The conditions under which greedy layer-local contrastive training converges to useful representations, and how the split topology affects convergence rate, are open questions. Connecting SCFF’s empirical performance to a formal analysis would strengthen the case for using it in safety-critical applications.

Conclusion

This project shows that the SCFF objective and split learning are a natural match. Because SCFF already trains each layer independently with a local loss, the physical split boundary introduces no new constraints and no accuracy penalty. The sequential multi-client relay extends this to distributed data while staying competitive in accuracy.

The key practical takeaway is that gradient-free split learning is achievable without sacrificing the learning signal: raw data stays on the client, no gradients cross any boundary, and the network still learns features that support 79–80% test accuracy on CIFAR-10 with a linear probe.

The repository is at <https://github.com/MShayan-Ejaz/SCFF-Split-Learning>.

References

- Martin Abadi, Andy Chu, Ian Goodfellow, et al. Deep learning with differential privacy. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 308–318, 2016.
- Yoshua Bengio, Pascal Lamblin, Dan Popovici, and Hugo Larochelle. Greedy layer-wise training of deep networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 19, 2007.
- Mathilde Caron, Hugo Touvron, Ishan Misra, et al. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9650–9660, 2021.

- Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 1597–1607, 2020.
- Jean-Bastien Grill, Florian Strub, Florent Alché, et al. Bootstrap your own latent: A new approach to self-supervised learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 21271–21284, 2020.
- Zecheng He, Tianwei Zhang, and Ruby B. Lee. Model inversion attacks against collaborative inference. In *Proceedings of the 35th Annual Computer Security Applications Conference (ACSAC)*, pages 148–162, 2019.
- Geoffrey Hinton. The forward-forward algorithm: Some preliminary investigations. *arXiv preprint arXiv:2212.13345*, 2022.
- Sai Praneeth Karimireddy, Satyen Kale, Mehryar Mohri, Sashank Reddi, Sebastian Stich, and Ananda Theertha Suresh. SCAFFOLD: Stochastic controlled averaging for federated learning. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 5132–5143, 2020.
- Alex Krizhevsky and Geoffrey Hinton. Learning multiple layers of features from tiny images. In *Technical Report, University of Toronto*, 2009.
- Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Alexander Smola, and Virginia Smith. Federated optimization in heterogeneous networks. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 2, pages 429–450, 2020.
- Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 1273–1282, 2017.
- Arild Nøkland and Lars Haakon Eidnes. Training neural networks with local error signals. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 4839–4850, 2019.
- Dario Pasquini, Giuseppe Ateniese, and Massimo Bernaschi. Unleashing the tiger: Inference attacks on split learning. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2113–2129, 2021.
- Chandra Thapa, Seyit Ragib Chhetri, Antonios Ignatiadis, et al. SplitFed: When federated learning meets split learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 8485–8493, 2022.
- Praneeth Vepakomma, Otkrist Gupta, Tristan Swedish, and Ramesh Raskar. Split learning for health: Distributed deep learning without sharing raw patient data. *arXiv preprint arXiv:1812.00564*, 2018.